

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №7

Выполнил:

Батуров Максим Дмитриевич

Группа: 6203-010302D

Оглавление

Задание №1	2
Задание №2	4
Задание №3	7

Задание №1

В интерфейс `TabulatedFunction` было добавлено наследование от `Iterable<FunctionPoint>`.

В `ArrayTabulatedFunction` и `LinkedListTabulatedFunction` реализованы анонимные классы-итераторы, работающие непосредственно с внутренними структурами данных.

Итератор для `ArrayTabulatedFunction` использует индексацию массива точек, а для `LinkedListTabulatedFunction` - обход узлов связного списка. Оба итератора возвращают новые экземпляры точек, что предотвращает нарушение инкапсуляции. Метод `remove()` выбрасывает исключение `UnsupportedOperationException`.

```
@Override
public Iterator<FunctionPoint> iterator() {
    return new Iterator<FunctionPoint>() {
        private int currentIndex = 0; 2 usages

        @Override
        public boolean hasNext() {
            return currentIndex < pointsCount;
        }

        @Override
        public FunctionPoint next() {
            if (!hasNext()) {
                throw new NoSuchElementException("нет следующего элемента");
            }
            // возвращаем копию для защиты инкапсуляции
            return new FunctionPoint(points[currentIndex++]);
        }

        @Override
        public void remove() {
            throw new UnsupportedOperationException("удаление не поддерживается");
        }
    };
}
```

```

// реализация итератора для поддержки for-each
@Override
public Iterator<FunctionPoint> iterator() {
    return new Iterator<FunctionPoint>() {
        private FunctionNode currentNode = head.getNext(); 4 usages

        @Override
        public boolean hasNext() {
            return currentNode != head;
        }

        @Override
        public FunctionPoint next() {
            if (!hasNext()) {
                throw new NoSuchElementException("нет следующего элемента");
            }

            FunctionPoint point = currentNode.getPoint();
            currentNode = currentNode.getNext();

            // возвращаем копию для защиты инкапсуляции
            return new FunctionPoint(point);
        }

        @Override
        public void remove() {
            throw new UnsupportedOperationException("удаление не поддерживается");
        }
    };
}
}

```

```

public class Main {
    public static void main(String[] args) {
        // ЗАДАНИЕ 1
        System.out.println("ИТЕРАТОРЫ:");
        FunctionPoint[] points = {
            new FunctionPoint(x: 0, y: 1),
            new FunctionPoint(x: 1, y: 2),
            new FunctionPoint(x: 2, y: 3),
            new FunctionPoint(x: 3, y: 4)
        };

        TabulatedFunction f = new ArrayTabulatedFunction(points);
        for (FunctionPoint p : f) {
            System.out.println(p);
        }
    }
}

```

```
ИТЕРАТОРЫ:  
(0.0; 0.0)  
(1.0714285714285714; 0.0)  
(2.142857142857143; 0.0)  
(3.2142857142857144; 0.0)  
(4.285714285714286; 0.0)  
(5.357142857142857; 0.0)  
(6.428571428571429; 0.0)  
(7.5; 0.0)  
(8.571428571428571; 0.0)  
(9.642857142857142; 0.0)  
(10.714285714285714; 0.0)  
(11.785714285714285; 0.0)  
(12.857142857142858; 0.0)  
(13.928571428571429; 0.0)  
(15.0; 0.0)
```

Задание №2

Определен интерфейс `TabulatedFunctionFactory` с тремя перегруженными методами `createTabulatedFunction()`, соответствующими конструкторам классов табулированных функций.

В каждом классе табулированных функций добавлен вложенный публичный класс-фабрика. Эти классы реализуют интерфейс фабрики и создают экземпляры соответствующих типов.

В классе `TabulatedFunctions` объявлено статическое поле типа `TabulatedFunctionFactory`, инициализированное фабрикой для `ArrayTabulatedFunction`. Добавлен метод `setTabulatedFunctionFactory()` для замены используемой фабрики во время выполнения программы.

```
package functions;  
  
public interface TabulatedFunctionFactory { no usages 2 implementations  
    // создает табулированную ф-цию с нулевыми значениями  
    TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount);  
  
    // создает табулированную ф-цию с заданными значениями в точках  
    TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values);  
  
    // создает табулированную ф-цию из массива точек  
    TabulatedFunction createTabulatedFunction(FunctionPoint[] points); no usages 2 implementations  
}
```

```
// класс-фабрика для создания ArrayTabulatedFunction
public static class ArrayTabulatedFunctionFactory implements TabulatedFunctionFactory { no usages

    @Override no usages
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) {
        return new ArrayTabulatedFunction(leftX, rightX, pointsCount);
    }

    @Override no usages
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) {
        return new ArrayTabulatedFunction(leftX, rightX, values);
    }

    @Override no usages
    public TabulatedFunction createTabulatedFunction(FunctionPoint[] points) {
        return new ArrayTabulatedFunction(points);
    }
}
```

```
// класс-фабрика для создания LinkedListTabulatedFunction
public static class LinkedListTabulatedFunctionFactory implements TabulatedFunctionFactory { no usages

    @Override no usages
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) {
        return new LinkedListTabulatedFunction(leftX, rightX, pointsCount);
    }

    @Override no usages
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) {
        return new LinkedListTabulatedFunction(leftX, rightX, values);
    }

    @Override no usages
    public TabulatedFunction createTabulatedFunction(FunctionPoint[] points) {
        return new LinkedListTabulatedFunction(points);
    }
}
```

```

public class TabulatedFunctions { 6 usages
    private static TabulatedFunctionFactory factory = new ArrayTabulatedFunction.ArrayTabulatedFunctionFactory(); 4

    // приватный конструктор
    private TabulatedFunctions() {} no usages

    // устанавливает фабрику для создания табулированных ф-ций
    public static void setTabulatedFunctionFactory(TabulatedFunctionFactory factory) { 2 usages
        TabulatedFunctions.factory = factory;
    }

    // создает табулированную ф-цию с нулевыми значениями (фабрика)
    public static TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) { no usages
        return factory.createTabulatedFunction(leftX, rightX, pointsCount);
    }

    // создает табулированную ф-цию с заданными значениями (фабрика)
    public static TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) { 1 usage
        return factory.createTabulatedFunction(leftX, rightX, values);
    }

    // создает табулированную ф-цию из массива точек (фабрика)
    public static TabulatedFunction createTabulatedFunction(FunctionPoint[] points) { 2 usages
        return factory.createTabulatedFunction(points);
    }
}

```

// ЗАДАНИЕ 2

```

System.out.println("\nФАБРИКА:");
Function cos = new Cos();
TabulatedFunction tf;

tf = TabulatedFunctions.tabulate(cos, leftX: 0, Math.PI, pointsCount: 11);
System.out.println(tf.getClass());

TabulatedFunctions.setTabulatedFunctionFactory(
    new LinkedListTabulatedFunction.LinkedListTabulatedFunctionFactory());
tf = TabulatedFunctions.tabulate(cos, leftX: 0, Math.PI, pointsCount: 11);
System.out.println(tf.getClass());

TabulatedFunctions.setTabulatedFunctionFactory(
    new ArrayTabulatedFunction.ArrayTabulatedFunctionFactory());
tf = TabulatedFunctions.tabulate(cos, leftX: 0, Math.PI, pointsCount: 11);
System.out.println(tf.getClass());

```

ФАБРИКА:

```

class functions.ArrayTabulatedFunction
class functions.LinkedListTabulatedFunction
class functions.ArrayTabulatedFunction

```

Задание №3

В классе `TabulatedFunctions` добавлены три перегруженных метода `createTabulatedFunction()`. Эти методы осуществляют поиск конструктора с соответствующими параметрами с помощью `getConstructor()` и создают объект через `newInstance()`.

Кроме методов создания, также добавлены перегруженные версии методов чтения из потоков:

`inputTabulatedFunction(Class, InputStream)` - чтение из байтового потока с указанием типа

`readTabulatedFunction(Class, Reader)` - чтение из символьного потока с указанием типа

При возникновении исключений в процессе рефлексии они перехватываются и преобразуются в `IllegalArgumentException`.

Также добавлен метод `tabulate()` с параметром класса, который использует рефлексия для создания табулированной функции заданного типа

```
public static TabulatedFunction createTabulatedFunction(Class<? extends TabulatedFunction> functionClass, no usages
                                                       double leftX, double rightX, int pointsCount) {
    try {
        // получаем конструктор с нужными параметрами
        java.lang.reflect.Constructor<? extends TabulatedFunction> constructor =
            functionClass.getConstructor(double.class, double.class, int.class);
        // создаем объект
        return constructor.newInstance(leftX, rightX, pointsCount);
    } catch (java.lang.reflect.InvocationTargetException e) {
        Throwable cause = e.getCause();
        if (cause instanceof RuntimeException) {
            throw (RuntimeException) cause;
        }
        throw new IllegalArgumentException("ошибка при создании объекта через рефлексия", e);
    } catch (Exception e) {
        throw new IllegalArgumentException("ошибка при создании объекта через рефлексия", e);
    }
}

// создает табулированную ф-цию с заданными значениями (рефлексия)
public static TabulatedFunction createTabulatedFunction(Class<? extends TabulatedFunction> functionClass, 1 usage
                                                       double leftX, double rightX, double[] values) {
    try {
        // получаем конструктор с нужными параметрами
        java.lang.reflect.Constructor<? extends TabulatedFunction> constructor =
            functionClass.getConstructor(double.class, double.class, double[].class);
        // создаем объект
        return constructor.newInstance(leftX, rightX, values);
    } catch (java.lang.reflect.InvocationTargetException e) {
        Throwable cause = e.getCause();
        if (cause instanceof RuntimeException) {...}
        throw new IllegalArgumentException("ошибка при создании объекта через рефлексия", e);
    } catch (Exception e) {
        throw new IllegalArgumentException("ошибка при создании объекта через рефлексия", e);
    }
}

// создает табулированную ф-цию из массива точек (рефлексия)
public static TabulatedFunction createTabulatedFunction(Class<? extends TabulatedFunction> functionClass, no usages
                                                       FunctionPoint[] points) {
    try {
        // получаем конструктор с нужными параметрами
        java.lang.reflect.Constructor<? extends TabulatedFunction> constructor =
            functionClass.getConstructor(FunctionPoint[].class);
        // создаем объект
        return constructor.newInstance((Object) points);
    } catch (java.lang.reflect.InvocationTargetException e) {
        Throwable cause = e.getCause();
        if (cause instanceof RuntimeException) {
            throw (RuntimeException) cause;
        }
        throw new IllegalArgumentException("ошибка при создании объекта через рефлексия", e);
    }
}
```

```
// табулирует ф-цию (фабрика)
public static TabulatedFunction tabulate(Function function, double leftX, double rightX, int pointsCount) {
    // проверка границ
    if (leftX < function.getLeftDomainBorder() || rightX > function.getRightDomainBorder()) {
        throw new IllegalArgumentException("границы табулирования выходят за область определения ф-ции");
    }

    // проверка минимального количества точек
    if (pointsCount < 2) {
        throw new IllegalArgumentException("количество точек должно быть не менее 2");
    }

    // проверка интервала
    if (leftX >= rightX) {
        throw new IllegalArgumentException("левая граница должна быть меньше правой");
    }

    // создаем массив значений ф-ции
    double[] values = new double[pointsCount];
    double step = (rightX - leftX) / (pointsCount - 1);

    // вычисляем значения ф-ции
    for (int i = 0; i < pointsCount; i++) {
        double x = leftX + i * step;
        values[i] = function.getFunctionValue(x);
    }

    // используем фабрику для создания табулированной ф-ции
    return createTabulatedFunction(leftX, rightX, values);
}
```



```

// вывод табулированной ф-ции в байтовый поток
public static void outputTabulatedFunction(TabulatedFunction function, OutputStream out) throws IOException {
    DataOutputStream dataOut = new DataOutputStream(out);

    // записываем кол-во точек
    dataOut.writeInt(function.getPointsCount());

    // записываем координаты всех точек
    for (int i = 0; i < function.getPointsCount(); i++) {
        dataOut.writeDouble(function.getPointX(i));
        dataOut.writeDouble(function.getPointY(i));
    }

    dataOut.flush();
}

// ввод табулированной ф-ции из байтового потока
public static TabulatedFunction inputTabulatedFunction(InputStream in) throws IOException {
    DataInputStream dataIn = new DataInputStream(in);

    // читаем кол-во точек
    int pointsCount = dataIn.readInt();

    // читаем координаты точек
    FunctionPoint[] points = new FunctionPoint[pointsCount];
    for (int i = 0; i < pointsCount; i++) {
        double x = dataIn.readDouble();
        double y = dataIn.readDouble();
        points[i] = new FunctionPoint(x, y);
    }

    // используем фабрику для создания табулированной ф-ции
    return createTabulatedFunction(points);
}

```

```

// запись ф-ции в символьный поток
public static void writeTabulatedFunction(TabulatedFunction function, Writer out) throws IOException {
    PrintWriter writer = new PrintWriter(out);

    // записываем кол-во точек
    writer.print(function.getPointsCount());
    writer.print(" ");

    for (int i = 0; i < function.getPointsCount(); i++) {
        writer.print(function.getPointX(i));
        writer.print(" ");
        writer.print(function.getPointY(i));
        if (i < function.getPointsCount() - 1) {
            writer.print(" ");
        }
    }

    writer.flush();
}

```

```

public static TabulatedFunction readTabulatedFunction(Reader in) throws IOException {
    if (in == null) {
        throw new IllegalArgumentException("передан нулевой поток ввода");
    }

    StreamTokenizer tokenizer = new StreamTokenizer(in);
    tokenizer.parseNumbers();

    // считываем число точек
    int tokenType = tokenizer.nextToken();
    if (tokenType != StreamTokenizer.TT_NUMBER) {
        throw new IOException("не найдено количество точек");
    }
    int count = (int) tokenizer.nval;

    // создаем массив для хранения точек
    FunctionPoint[] functionPoints = new FunctionPoint[count];

    // считываем пары
    for (int i = 0; i < count; i++) {
        // считываем x координату
        tokenType = tokenizer.nextToken();
        if (tokenType != StreamTokenizer.TT_NUMBER) {
            throw new IOException("отсутствует x координата для точки " + i);
        }
        double xCoord = tokenizer.nval;

        // считываем y координату
        tokenType = tokenizer.nextToken();
        if (tokenType != StreamTokenizer.TT_NUMBER) {
            throw new IOException("отсутствует y координата для точки " + i);
        }
        double yCoord = tokenizer.nval;

        // создаем точку с полученными координатами
        functionPoints[i] = new FunctionPoint(xCoord, yCoord);
    }

    // используем фабрику для создания табулированной ф-ции
    return createTabulatedFunction(functionPoints);
}

```

```
// ЗАДАНИЕ 3
System.out.println("\nРЕФЛЕКСИЯ:");

// тест 1: создание ArrayTabulatedFunction через рефлексию
f = TabulatedFunctions.createTabulatedFunction(
    ArrayTabulatedFunction.class, leftX: 0, rightX: 10, pointsCount: 3);
System.out.println(f.getClass());
System.out.println(f);

// тест 2: создание ArrayTabulatedFunction с массивом значений
f = TabulatedFunctions.createTabulatedFunction(
    ArrayTabulatedFunction.class, leftX: 0, rightX: 10, new double[] {0, 10});
System.out.println(f.getClass());
System.out.println(f);

// тест 3: создание LinkedListTabulatedFunction из массива точек
f = TabulatedFunctions.createTabulatedFunction(
    LinkedListTabulatedFunction.class,
    new FunctionPoint[] {
        new FunctionPoint(x: 0, y: 0),
        new FunctionPoint(x: 10, y: 10)
    }
);
System.out.println(f.getClass());
System.out.println(f);

// тест 4: табулирование через рефлексию
f = TabulatedFunctions.tabulate(
    LinkedListTabulatedFunction.class, new Sin(), leftX: 0, Math.PI, pointsCount: 11);
System.out.println(f.getClass());
System.out.println(f);
```

```
РЕФЛЕКСИЯ:
class functions.ArrayTabulatedFunction
{(0.0; 0.0), (5.0; 0.0), (10.0; 0.0)}
class functions.ArrayTabulatedFunction
{(0.0; 0.0), (10.0; 10.0)}
class functions.LinkedListTabulatedFunction
{(0.0; 0.0), (10.0; 10.0)}
class functions.LinkedListTabulatedFunction
{(0.0; 0.0), (0.3141592653589793; 0.3090169943749474), (0.6283185307179586; 0.5877852522924731),
Process finished with exit code 0
```